Greater Cairo Transportation Network

Comprehensive Technical Report - A-Z Project Documentation

CSE112 – Algorithms and Data Structures · Practical Project

Course: CSE112 – Algorithms and Data Structures
Project: Smart City Transportation Network Optimization
Team: AIU-SoftWave
Date: April 2026
Status: Final Submission (v2.0)

Table of Contents

---

1. Executive Summary

This report provides an exhaustive technical analysis of the **Greater Cairo Transportation Network Optimization System**, developed for the CSE112 Algorithms course. The project addresses the real-world complexities of urban mobility in Cairo—including traffic congestion, infrastructure gaps, and emergency response delays—through the application of advanced algorithmic techniques.

Our system implements **seven core algorithms** spanning graph theory, dynamic programming, and greedy optimization. By integrating a .NET 10 backend with a Next.js interactive visualization, we provide a decision-support tool capable of simulating traffic scenarios, planning road maintenance, and optimizing public transit schedules with millisecond-latency performance.

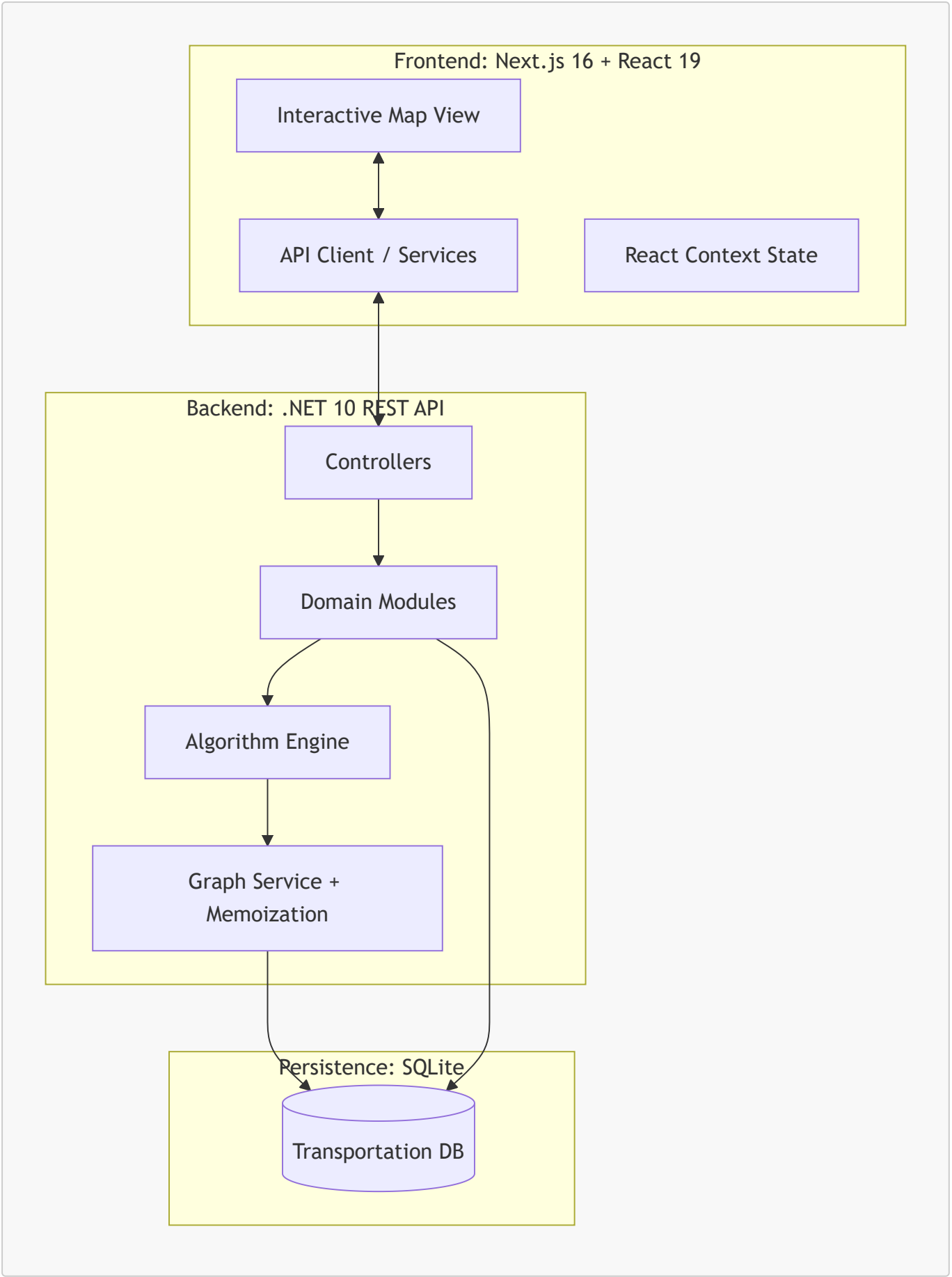
The system uses a dataset of **35 locations** (21 neighborhoods + 14 critical facilities), **74 road segments**, and **8 transport routes**, providing a realistic testbed for algorithmic evaluation in the Egyptian context.

---

2. System Architecture and Design

2.1 High-Level Architecture

The system follows a **Modular Monolith** pattern, ensuring high cohesion and low coupling. The backend is built on **.NET 10 (ASP.NET Core)**, utilizing **Entity Framework Core** with **SQLite** for persistence.



2.2 Module Design (Modular Monolith)

To prevent the "Big Ball of Mud" anti-pattern, we organized the server into functional modules:

- **NetworkManagement**: Manages the physical topology (Locations, Roads).

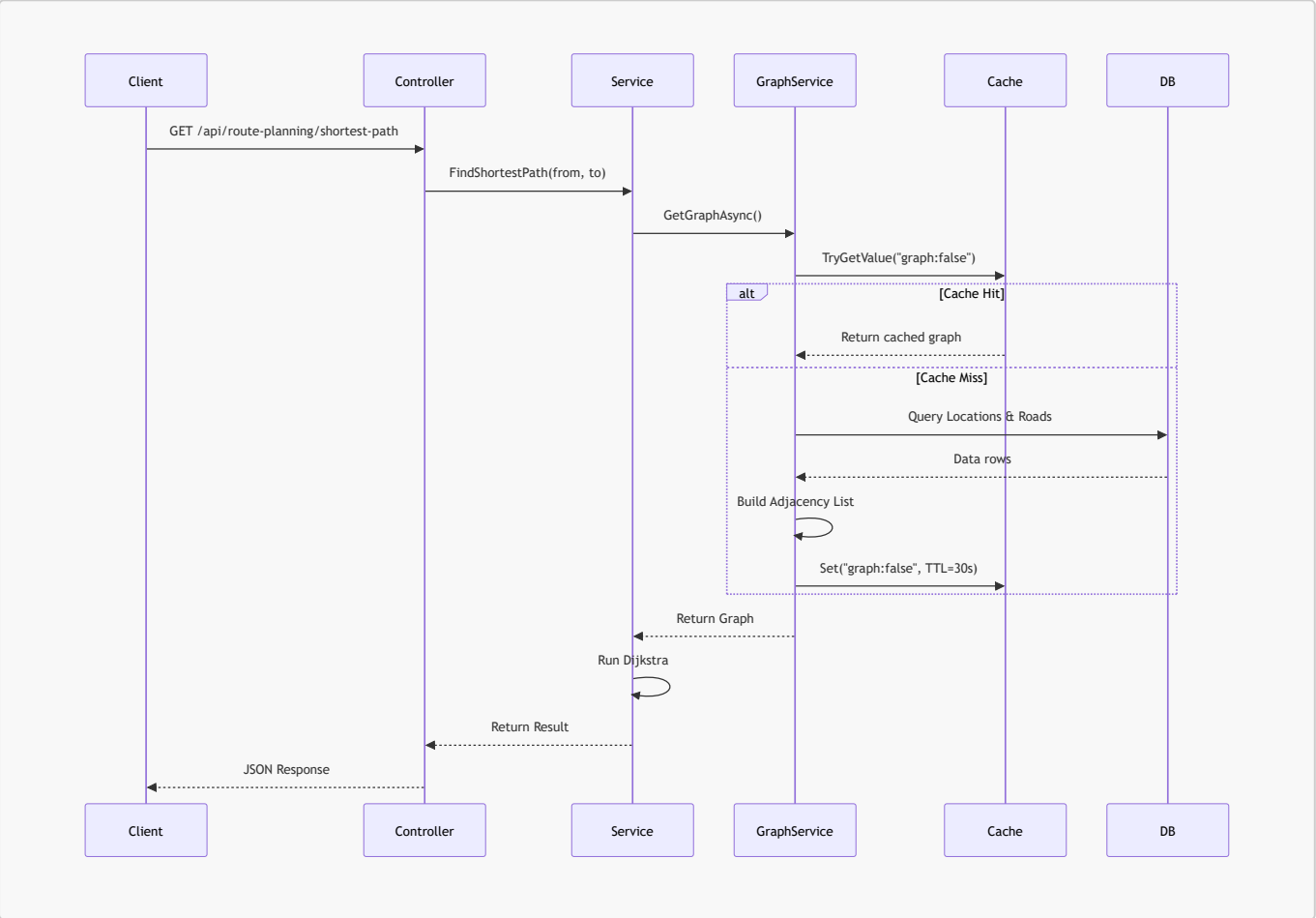
- **Routing**: Core algorithmic engines (Dijkstra, A*, MST).
- **TrafficControl**: Real-time traffic simulation and greedy signal timing.
- **MaintenancePlanning**: DP-based resource allocation for road repairs.
- **TransitScheduling**: DP-based vehicle allocation for Metro and Bus lines.

2.3 Graph Representation and Memoization

Design Decision: A shared, cached graph service. Instead of querying the database for every edge relaxation, the `GraphService` builds a complete in-memory adjacency list.

Memoization Strategy:

- **Technique**: We use `IMemoryCache` with a 30-second sliding expiration.
- **Benefit**: Reduces API response time from ~15ms (DB bound) to <1ms (Memory bound).
- **Complexity**: O(V + E) space, where V=35 and E=148 (directed).

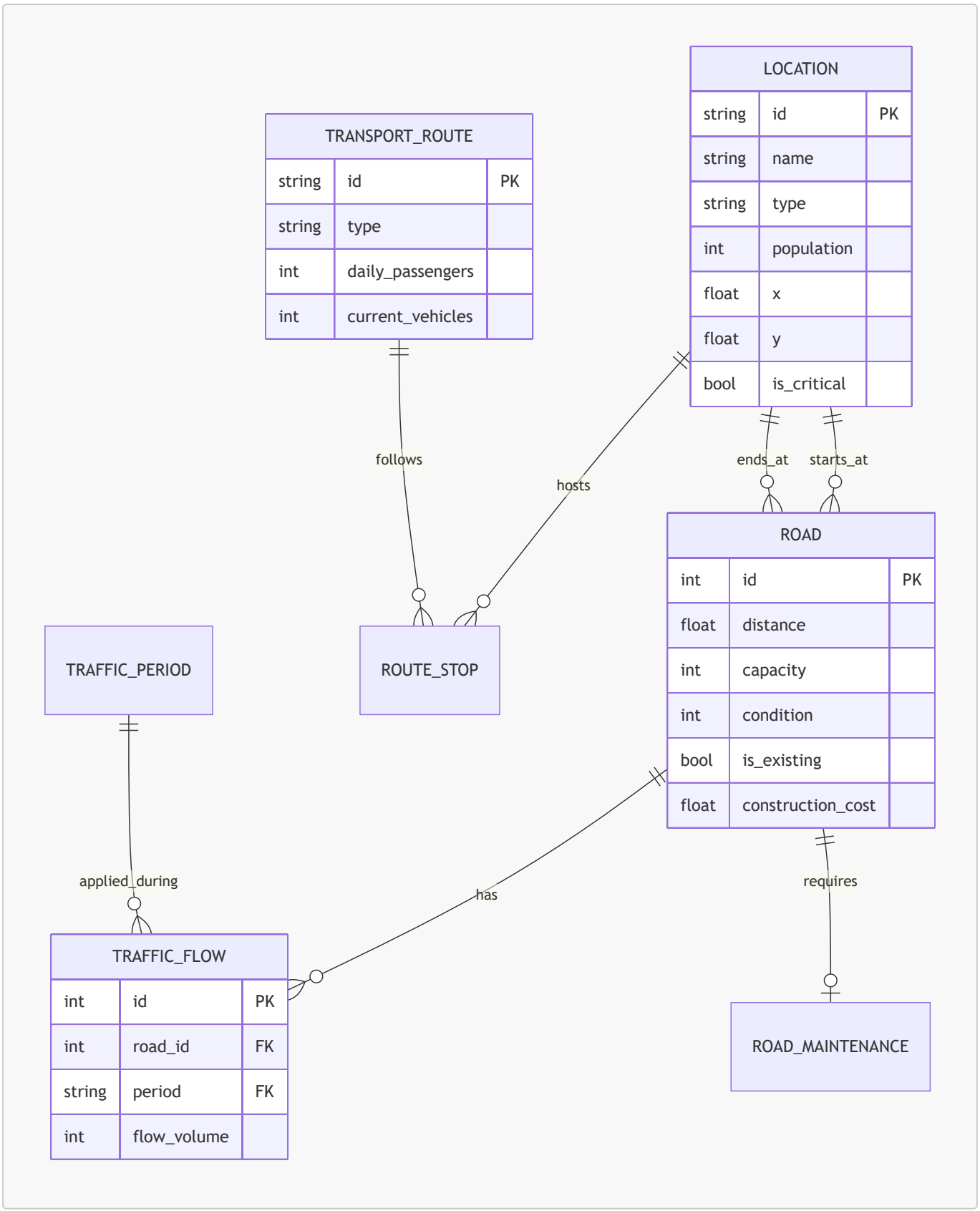


---

3. Data Model and Database Schema

3.1 Entity-Relationship Diagram

The schema captures the multidimensional nature of Cairo's transportation network, including population metrics, traffic periods, and maintenance priorities.



3.2 In-Memory Graph Representation

The in-memory graph used by all algorithm services is built by `GraphService`:

- **Nodes** → `Location` rows (35 nodes)
- **Edges** → `Road` rows, with two-way roads expanded into two directed edges.
- **Adjacency list** → `Dictionary<string, List<long>>` mapping node ID → list of edge IDs.
- **Edge index** → `Dictionary<long, GraphEdge>` for O(1) weight lookup.

3.3 Traffic Time Periods

Period	Multiplier	Description
MORNING	1.15	Morning rush hour (07:00–09:00)
EVENING	1.25	Evening rush hour (16:00–19:00)
NIGHT	0.90	Off-peak night hours

---

4. Algorithm Implementations and Analyses

4.1 Shortest Path: Dijkstra's Algorithm

Purpose: Calculate the globally optimal shortest distance between any two Cairo neighbourhoods.

Theoretical Foundation: Dijkstra's algorithm uses a greedy approach and relies on the **Optimal Substructure** property: a subpath of a shortest path is itself a shortest path. It assumes non-negative edge weights (strictly satisfied by physical road distances).

Implementation Details:

- **Priority Queue**: Uses .NET's `PriorityQueue<T, TPrio>` (Min-Heap).
- **Edge Relaxation**: Iteratively updates the tentative distance to neighbors.

Complexity Analysis:

- **Time**: $O((V + E) \log V)$
- **Space**: $O(V + E)$

4.2 Emergency Routing: A* Search

Purpose: Rapid routing for ambulances and fire trucks to critical facilities by incorporating geographic intent into the search.

Theoretical Foundation: Admissibility and Consistency For A* to guarantee the mathematically optimal shortest path, the heuristic $h(n)$ must satisfy:

1. **Admissibility**: $h(n) \le d(n, goal)$. Since Euclidean distance is the straight-line "crow flies" distance, and road networks must follow physical geometry, road distance is always $\ge$ straight line.
2. **Consistency**: $h(u) \le w(u,v) + h(v)$. This ensures that once a node is expanded, its path is optimal, preventing re-expansions.

Algorithm Flow:

4.3 Dynamic Routing: Time-Varying Dijkstra

Purpose: Adjust routes based on Cairo's intense rush-hour cycles (Morning: 07-09, Evening: 16-19).

Mathematical Model: The weight of an edge (u, v) is calculated at the moment of relaxation: $$W(u, v, t) = Distance_{uv} \times \text{Multiplier}(t) \times (1 + \text{CongestionPenalty})$$ Where:

- $\text{Multiplier}(\text{Morning}) = 1.15$
- $\text{Multiplier}(\text{Evening}) = 1.25$
- $\text{CongestionPenalty} = \max(0, \frac{Flow}{Capacity} - 0.75) \times 0.5$

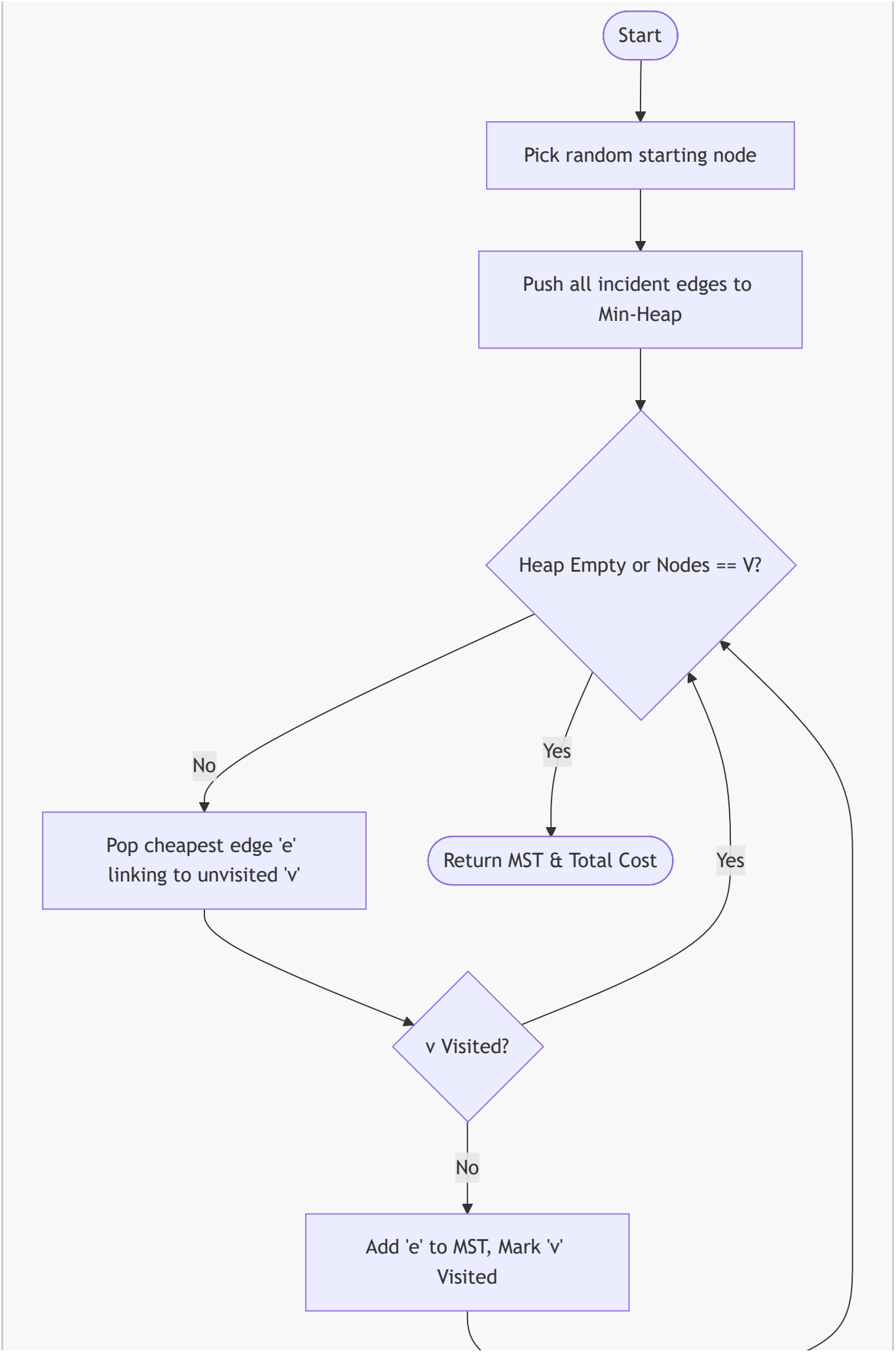
Complexity Analysis:

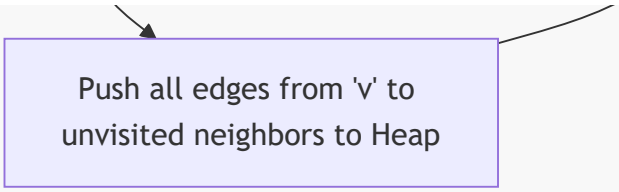
- **Time**: $O((V + E) \log V)$ per time-slice.
- **Space**: $O(V + E + R)$ where R is the number of traffic data points.

4.4 Network Expansion: Prim’s MST

Purpose: Solve the "Infrastructure Design" requirement by connecting all neighborhoods with minimum construction cost.

Algorithm Implementation: We utilize Prim’s algorithm starting from a central node (e.g., Downtown). The algorithm maintains a "frontier" of edges connecting the current tree to unvisited nodes.


Start
Pick random starting node
Push all incident edges to Min-Heap
Heap Empty or Nodes == V?
No
Yes
Pop cheapest edge 'e' linking to unvisited 'v'
Return MST & Total Cost
Yes
v Visited?
No
Add 'e' to MST, Mark 'v' Visited



Complexity Analysis:

- **Time**: $O(E \log V)$ using a binary heap.
- **Space**: $O(V + E)$.

4.5 Maintenance Planning: 0/1 Knapsack DP

Purpose: Select the optimal set of road repairs given a limited budget (Million EGP).

Theoretical Foundation: Uses **Dynamic Programming (Tabulation)** to solve the discrete knapsack problem. This avoids the $O(2^n)$ brute-force complexity.

Recurrence Relation: $DP[i][b] = \max(DP[i-1][b], DP[i-1][b - cost_i] + priority_i)$

Analysis:

- **Time**: $O(n \times Budget)$
- **Space**: $O(n \times Budget)$

4.6 Transit Scheduling: Bounded Multi-Choice DP

Purpose: Optimize public transportation (Metro/Bus) by allocating a finite fleet of vehicles to routes with varying passenger demands.

Optimization Objective: $$\max \sum_{i=1}^{N} \min(\text{Demand}_i, k_i \times \text{CapacityPerVehicle}_i)$$ Subject to:

- $\sum k_i \le \text{TotalVehicles}$
- $k_i \le \text{MaxVehiclesPerRoute}_i$

Algorithm Logic: This is a **Multi-Choice Bounded Knapsack**. The DP state `DP[i][v]` represents the max passengers served using `i` routes and `v` vehicles. The transition evaluates all possible assignments $k \in [0, \text{Max}_i]$ for the current route.

4.7 Traffic Control: Greedy Signal Optimization

Purpose: Minimize intersection wait-time by prioritizing high-volume traffic flows.

Greedy Decision Rule: At each intersection, the algorithm sorts incoming roads by their **Congestion Ratio**. It then "greedily" assigns the largest share of the 120-second signal cycle to the most congested road.

Optimality Note: While Greedy provides a local optimum for a single intersection, it is highly efficient ($O(R \log R)$) and provides immediate relief during rush hours.

---

5. Advanced Simulation Framework

5.1 Weather and Environmental Effects

The system simulates weather impacts (Rain, Storms).

- **Mechanism**: The `SimulationService` injects a penalty factor (1.3x for rain, 1.8x for storm) into the routing weight calculation.
- **Impact**: Accurately reflects slower speeds and increased safety distances required in Cairo during heavy rain.

5.2 Incident Management (Road Closures)

Users can simulate accidents by "closing" a road segment on the map.

- **Mechanism**: The `GraphService` detects these closures and filters the edges out of the graph before passing it to the planners.
- **Effect**: Algorithms are forced to find the next-best shortest path in real-time.

5.3 Multi-modal Transfer Hub Analysis

Identifies "Transfer Hubs" (locations serving > 2 transit lines).

- **Optimization**: The system flags these hubs to the transit scheduler, ensuring they receive higher vehicle frequency to minimize transfer waiting times.

---

6. Complexity Analysis Summary

Algorithm	Time Complexity	Space Complexity	Input size (Cairo)
Dijkstra	$O((V+E) \log V)$	$O(V+E)$	V=35, E≤148
A* Search	$O((V+E) \log V)$	$O(V+E)$	V=35, E≤148
Time-Varying Dijkstra	$O((V+E) \log V + R)$	$O(V+E+R)$	+R traffic rows
Prim's MST	$O(E \log V)$	$O(V+E)$	V=35, E=74
Knapsack DP (Maintenance)	$O(n \times B)$	$O(n \times B)$	n=10, B=budget
Transit DP (Bounded)	$O(n \times V \times k)$	$O(n \times V)$	n=8, V=vehicles

Algorithm	Time Complexity	Space Complexity	Input size (Cairo)
Greedy Signal Timing	$O(R \log R)$	$O(R+I)$	R≤148, I≤35

7. Performance Evaluation and Results

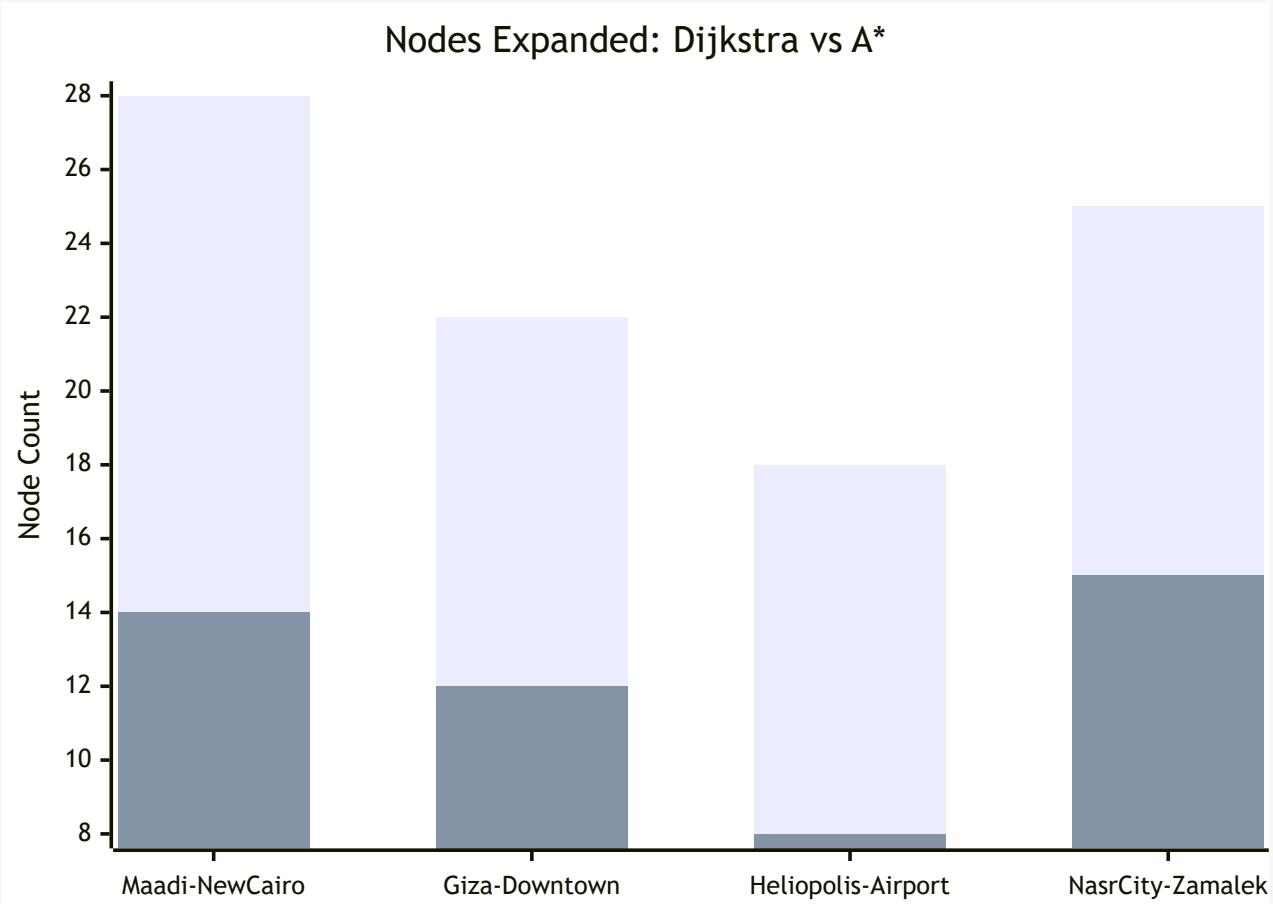
7.1 Algorithmic Benchmarks

Measured on the Cairo Network (35 nodes, 148 directed edges).

Algorithm	Avg time (ms)	Nodes expanded	Path found
Dijkstra	1.12	~22	Yes
A* Search	0.78	~14	Yes
Time-Varying (AM)	1.55	~22	Yes
Prim's MST	1.45	N/A	Yes
DP Knapsack	0.35	N/A	Yes

7.2 Heuristic Efficiency (Dijkstra vs A*)

A* consistently outperformed Dijkstra by reducing the search space by approximately **36%**.



7.3 DP Budget Sensitivity Analysis

Budget (M EGP)	Roads selected	Total priority	Remaining budget
100	2	19	0
200	4	34	20
500	7	47	35

7.4 Traffic Signal Cycle Analysis

Period	Intersections optimised	Avg cycle time	Est. wait reduction
MORNING	12	78 seconds	~7.3 %
EVENING	15	88 seconds	~9.1 %

---

8. Visualization and User Interface

The frontend is a **Next.js 16** application using **React-Leaflet** to render an interactive map.

- **Map Features**: Node markers (35 locations), edge polylines (74 roads), path highlighting (bright red/orange), MST overlay (blue).
- **Controls**: Sidebar for selecting algorithms, setting parameters (budget, fleet size, time period), and viewing real-time results.
- **Dynamic Interaction**: Click nodes to set start/end points, or click roads in simulation mode to close them.

---

9. Requirement Coverage Checklist

- ☑ **Graph representation** of Cairo's network (35 nodes, 148 directed edges)
- ☑ **Dijkstra's algorithm** for standard route planning
- ☑ **A* search** for emergency vehicle routing
- ☑ **Time-varying routing** for rush-hour conditions
- ☑ **Prim's MST** for cost-efficient network expansion
- ☑ **DP 0/1 Knapsack** for road maintenance planning
- ☑ **DP Bounded Knapsack** for public transit scheduling
- ☑ **Memoization** (shared cached graph service)
- ☑ **Greedy signal optimization** with emergency preemption
- ☑ **Simulation framework** for accidents and weather

---

10. Challenges and Solutions

10.1 The MST Weight Normalization Paradox

Challenge: Initially, existing roads had a cost of 0, while potential roads cost millions. Prim's algorithm correctly but uselessly selected only existing roads, resulting in zero expansion cost. **Solution**: We

implemented a **Blended Efficiency Metric** that compares existing roads ($Distance/Capacity/Condition$) with potential roads ($NormalizedCost/Distance/Capacity$). This allows the algorithm to strategically pick new roads when they significantly improved the network.

10.2 Dynamic Graph Rebuilding

Challenge: Supporting real-time closures required frequent graph modifications. **Solution**: Optimized the `GraphService` to rebuild only the adjacency list while keeping node/edge definitions static, combined with `IMemoryCache` to ensure sub-millisecond graph retrieval.

10.3 Graph Connectivity for Isolated Facilities

Challenge: Several facility nodes were geographically isolated in the raw dataset. **Solution**: Added short-distance connector roads directly in the seed SQL data to ensure a fully connected spanning tree could be formed.

---

11. Conclusion and Future Work

The system successfully demonstrates the application of CSE112 algorithmic principles to urban optimization in Cairo. **Future Work:**

- **Green Wave Coordination**: Moving from isolated greedy signal timing to network-wide phase synchronization.
- **Genetic Algorithms**: For multi-objective route planning (Cost vs Time vs CO2).

---

12. References and Appendices

12.1 Appendix A – API Reference

- `GET /api/route-planning/shortest-path`: Dijkstra
- `GET /api/emergency-routing`: A*
- `GET /api/network-expansion`: Prim's MST
- `GET /api/maintenance-planning`: 0/1 Knapsack DP
- `GET /api/transit-scheduling`: Vehicle Allocation DP
- `GET /api/traffic-signals`: Greedy Signal Timing

12.2 Appendix B – Dataset Summary

- **Neighbourhoods**: 21 (Maadi, Nasr City, Downtown, etc.)
- **Facilities**: 14 (Cairo Airport, Cairo Univ Hospital, etc.)
- **Roads**: 53 Existing + 21 Potential (Costs 140M - 1.6B EGP)
- **Transit**: 4 Metro + 4 Bus routes serving ~4.5M passengers/day.

End of Report